

BÁO CÁO TỔNG KẾT BÀI TẬP 5: KIỂM THỬ HIỆU NĂNG ỨNG DỤNG WEB (MAIN REPORT)

Môn học: Kiểm thử phần mềm (Software Testing)

Trường: Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS)

Mã bài tập (Exercise ID): HW05-AI

Họ và tên sinh viên: BaoBeiii (Lê Quốc Bảo)

Mã số sinh viên (MSSV): 23127327

Email sinh viên: 23127327@student.hcmus.edu.vn

Điểm tự đánh giá (Self-Assessed Grade): 100 / 100 Điểm

Tên tệp nộp bài quy định: 23127327_HW05_AI_Performance_100.zip

Môi trường máy trạm đo đạc: DESKTOP-BEIZU (Node.js v24.11.0, k6 v2.1.0, Windows 11)

Hệ thống kiểm thử (SUT): EShop Application (<https://github.com/ttbhanh/eshop-sut.git>)

[!IMPORTANT]

LIÊN KẾT VIDEO DEMO BÁO CÁO (YOUTUBE DEMO LINKS)

-  Video Demo Task 1 (Đo tải k6, Bẫy 7 bug SUT, Phản biện AI & HTML Dashboard): <https://youtu.be/leNk4TxJ1D4>
-  Video Demo Agent Skill (Đóng gói Agent Skill Antigravity, CLI Metrics Gate & CI/CD): <https://youtu.be/vSyRLXkW-7k>

MỤC LỤC

- Tổng Quan Hệ Thống & Phạm Vi Kiểm Thử (SUT & Scope)
- Thiết Kế Kịch Bản Kiểm Thử Bằng AI & Tham Số Hóa (Task 1)
- Phản Biện Của Con Người & Khắc Phục Lỗi Kịch Bản AI (Human Review Task 1)
- Báo Cáo Bẫy 7 Lỗi Hệ Thống Của SUT (Bug Hunting & SUT Defect Analysis)
- Kết Quả Thực Thi Đo Tải Thực Tế & Bằng Chứng Tài Nguyên (Execution Evidence)
- Phân Tích Kết Quả, Bắt Lỗi Suy Diễn AI & 4 Giải Pháp Tối Ưu Kiến Trúc (Task 2)
- Đề Xuất Mô Hình Kiểm Thử Hiệu Năng Liên Tục & Đóng Gói Agent Skill (Task 3)

- Đoạn Văn Phê Bình AI (AI Critique - Bắt Buộc Theo Mục 10)
- Bảng Đánh Giá Điểm Số Theo Mẫu Mục 15 (Assessment Template)
- Phụ Lục: Danh Mục Tập Bàn Giao & Trích Lục Git Commit Log

1. TỔNG QUAN HỆ THỐNG & PHẠM VI KIỂM THỬ (SUT & SCOPE)

1.1 Hệ Thống Kiểm Thử (SUT)

Hệ thống **EShop SUT** là ứng dụng thương mại điện tử viết trên nền tảng **Node.js Express** và lưu trữ dữ liệu bằng **SQLite3** (`database.sqlite`). Ứng dụng gồm các nhóm tính năng (Feature Pools):

- Pool A:** Xác thực người dùng (FR-01..FR-04), Danh mục & Sản phẩm (FR-05..FR-06).
- Pool B:** Giỏ hàng (FR-07), Thanh toán (FR-08), Khuyến mãi (FR-09), Quản lý trạng thái đơn hàng (FR-10), Lịch sử mua hàng (FR-11).
- Pool C & D:** Quản trị Web Admin (FR-12..FR-19) và Mobile App APIs.

1.2 Lựa Chọn 3 Nhóm Endpoint & Chu Trình Người Dùng Khép Kín (E2E Workflow)

Tuân thủ Mục 5 của đề bài, kịch bản bao phủ đầy đủ cả 3 nhóm endpoint trọng yếu:

- Auth-heavy:** `POST /api/login` (Xử lý băm mật khẩu bcrypt, cấp phát JWT và cơ chế bảo mật tự động khóa tài khoản FR-02).
- Read-heavy:** `GET /api/products` và `GET /api/products/:id` (Quét dữ liệu danh mục sản phẩm và chi tiết sản phẩm).
- Transactional:** `POST /api/apply-coupon`, `POST /api/checkout`, `POST /api/coupon-usage`, `GET /api/orders/my-orders` (Ghi đơn hàng, cập nhật số lần dùng mã khuyến mãi, truy vấn lịch sử).

Chu trình người dùng thực tế được mô phỏng: `01_Login` $\rightarrow$ `02_BrowseProducts` $\rightarrow$ `03_ProductDetail` $\rightarrow$ `04_ApplyCoupon` $\rightarrow$ `05_Checkout` $\rightarrow$ `06_CouponUsage` $\rightarrow$ `07_MyOrders`

2. THIẾT KẾ KỊCH BẢN KIỂM THỬ BẰNG AI & THAM SỐ HÓA (TASK 1)

Theo chiến lược AI-First, sinh viên đã dẫn dắt AI thiết kế 4 kịch bản kiểm thử tải đáp ứng chuẩn quy ước tên file `{StudentID}_{ScenarioType}_20260902` :

2.1 Ma Trận Cấu Hình Kịch Bản Tải

Kịch Bản (Scenario)	Mục Tiêu Kỹ Thuật	Tệp Kịch Bản k6 / JMeter	Cấu Hình Tải (Virtual Users & Stages)
1. Load Testing	Đánh giá tải phân vị \$p95\$ ở giờ cao điểm	<code>scripts/23127327_Load_20260902.js</code> <code>scripts/23127327_Load_20260902.jmx</code>	Ramp-up \$0 \rightarrow 50\text{VUs}\$ (2m) \$\rightarrow\$ Hold 50 VUs (5m) \$\rightarrow\$ Ramp-down (1m). Tổng: 8 phút.
2. Stress Testing	Xác định Điểm gãy (Breaking Point)	<code>scripts/23127327_Stress_20260902.js</code> <code>scripts/23127327_Stress_20260902.jmx</code>	Tăng tải bậc thang: 10, 30, 70, 120, 200 VUs (mỗi bậc 45s). Tổng: ~4 phút.
3. Spike Testing	Mô phỏng Flash Sale, đo Thời gian hồi phục	<code>scripts/23127327_Spike_20260902.js</code> <code>scripts/23127327_Spike_20260902.jmx</code>	5 VUs (30s) \$\rightarrow\$ Đột biến 150 VUs (10s) \$\rightarrow\$ Giữ 150 VUs (30s) \$\rightarrow\$ Hạ 5 VUs (10s) \$\rightarrow\$ Đo hồi phục (30s).
4. Endurance Test	Đo độ ổn định dài hạn & rò rỉ bộ nhớ	<code>scripts/23127327_Endurance_20260902.js</code>	Duy trì liên tục 30 VUs trong 15 phút (900 giây).

2.2 Tham Số Hóa Dữ Liệu Thực Tế Bằng CSV (Data-Driven Testing)

- `data/users.csv` : Chứa 52 tài khoản (admin, test user, user1..user50 và tài khoản test lỗi mật khẩu logout).
- `data/products.csv` : Chứa 15 sản phẩm với các dải giá từ 65.000đ đến 1.200.000đ.
- `data/orders.csv` : 20 bản ghi đơn hàng mẫu với địa chỉ giao hàng và mã coupon (`SAVE10` , `BIGBUY` , `VIP100`).

2.3 Áp Dụng 3 Dạng Báo Cáo / Listener Khác Biệt (Distinct Report Views)

- Kịch bản Load: Xuất **View Results Tree** trên JMeter và **Interactive HTML Dashboard** trên k6 (`results/load/summary.html`).
- Kịch bản Stress: Xuất **Summary Report** trên JMeter và **Structured Metrics JSON** trên k6 (`results/stress/metrics.json`).

- Kịch bản Spike: Xuất **Aggregate Report** trên JMeter và **Raw Metrics CSV / JTL** (`results/spike/raw_spike.jtl`).

3. PHẢN BIỆN CỦA CON NGƯỜI & KHẮC PHỤC LỖI KỊCH BẢN AI (HUMAN REVIEW TASK 1)

Chi tiết đối chiếu xem tại [Human_Review_Report.md](#).

Khi sinh viên rà soát bản sinh kịch bản ban đầu của AI, đã phát hiện **6 nhóm sai sót nghiêm trọng**:

6 NHÓM LỖI LỚN CỦA BẢN SINH KỊCH BẢN BAN ĐẦU CỦA AI		
Lỗi Ban Đầu Của AI	Nguyên Nhân Gốc Rễ (Root Cause Dimension)	Cách Con Người
1. Đoán sai 3 Endpoints (/api/orders thay vì /api/checkout)	Prompt Quality & Thiên kiến mẫu REST (LLM hallucinated generic REST endpoints)	Soi server.js /api/checkout /api/orders/m
2. Think-time tính phẳng sleep(1) cố định	Model Limitations (Thiếu kiến thức thực tế về người dùng)	Đổi sang ngẫu sleep(Math.ra
3. Assertion nông cạn (Chỉ check status 200)	Prompt Quality & Thiếu chiều sâu (Không đọc logic backend trả về)	Bổ sung Deep bắt BUG-01, B
4. Bỏ sót cơ chế FR-02 Account Lockout	Endpoint Characteristics (Không biết lockout khóa sau 3 lần sai)	Nạp 52 users tạo script re
5. Thiếu Tag phân tích trễ p95 từng bước	Model Limitations (Không biết phân tích p95 trong k6)	Gán tag name HTTP request
6. Thiếu báo cáo đồ họa HTML Dashboard	Môi trường chạy Offline (Thư viện CDN online bị lỗi trong k6)	Đóng gói k6-r xuất summary.

Minh Họa Khối Diff Sửa Đổi Kịch Bản (Before vs After)

```
--- a/AI_Generated_Script.js
+++ b/Human_Refined_Script.js
@@ -10,7 +10,7 @@
  export default function () {
-    // AI LỖI: Think time tính phẳng gây xung kích đồng bộ
-    sleep(1);
+    // CON NGƯỜI SỬA: Think time ngẫu nhiên 1s - 3s mô phỏng người thật
+    sleep(Math.random() * 2 + 1);

-    // AI LỖI: Đoán sai endpoint đặt hàng
-    let res = http.post('http://localhost:3000/api/orders', payload);
+    // CON NGƯỜI SỬA: Endpoint thực tế của SUT là /api/checkout
+    let res = http.post('http://localhost:3000/api/checkout', payload, { tags: { nam

-    // AI LỖI: Assertion nông cạn chỉ check 200
-    check(res, { 'status is 200': (r) => r.status === 200 });
+    // CON NGƯỜI SỬA: Deep Assertions bắt lỗi logic và sai kiểu dữ liệu
+    check(res, {
+      'Checkout status is 200': (r) => r.status === 200,
+      'Checkout returns orderId': (r) => r.json('orderId') !== undefined,
+      'Price is number (Detect BUG #3)': (r) => typeof r.json('price') === 'number
+    });
```

4. BÁO CÁO BẦY 7 LỖI HỆ THỐNG CỦA SUT (BUG HUNTING & SUT DEFECT ANALYSIS)

Chi tiết xem tại [bug_reports.md](#) và [test_cases.md](#).

Sinh viên đã rà soát toàn bộ 573 dòng mã nguồn `eshop-sut/backend/server.js`, phát hiện và cài bẫy thành công 7 lỗi hệ thống:

1. **BUG-01 (Critical - `server.js:398`)**: Sai công thức tính giảm giá phần trăm `total_amount * (1 - discount_value)`, khiến số tiền giảm bị âm và tổng thanh toán bị đội giá gấp 10 lần. Kịch bản k6 đã bắt được 59 lần lỗi này.
2. **BUG-02 (High - `server.js:144`)**: Lỗi hỏng SQL Injection khi tìm kiếm sản phẩm. Nối trực tiếp chuỗi `${searchQuery}` khiến backend crash vắng mã HTML 500 khi gặp ký tự nháy đơn.
3. **BUG-03 (Medium - `server.js:162`)**: Ép kiểu giá sản phẩm thành String đối với các ID chẵn (`if (row.id % 2 === 0) row.price = row.price.toString()`). Kịch bản k6 đã bắt

được 4.099 lần lỗi này.

4. **BUG-04 (High - `server.js:54-62`)**: Cơ chế Account Lockout (FR-02) cộng sai số lần thử (`+2` thay vì `+1`) và khóa tới 3 phút (180s) thay vì 30s.
5. **BUG-05 (High - `server.js:296-308`)**: Checkout không xác thực lại giỏ hàng và tổng tiền. Tin tưởng 100% `total_amount` từ client, cho phép chỉnh sửa giá về 0đ.
6. **BUG-06 (Medium - `server.js:550-552`)**: Máy trạng thái đơn hàng (FR-10) cho phép chuyển phí lý từ `canceled` sang `delivered` (`if (currentStatus === "canceled" && status === "delivered") isValidTransition = true;`).
7. **BUG-07 (Critical - `server.js:387-454`)**: Lỗi hỏng Race Condition khi áp dụng mã giảm giá dưới tải đồng thời (thiếu Database Transaction), cho phép 1 user dùng vượt quá `max_uses_per_user` .

5. KẾT QUẢ THỰC THI ĐO TẢI THỰC TẾ & BẢNG CHỨNG TÀI NGUYÊN (EXECUTION EVIDENCE)

5.1 Bảng Số Liệu Đo Đặc Thực Nghiệm Toàn Diện

Toàn bộ 4 bài kiểm thử được thực thi trên máy trạm `DESKTOP-BEIZU` với SUT backend Node.js (PID: 13212):

Chỉ Số Đo Đặc	1. Load Testing	2. Stress Testing	3. Spike Testing	4. Endurance Testing
Cấu hình Virtual Users (VUs)	Peak 50 VUs	Bậc thang 10 \$\\rightarrow\$ 200 VUs	Đột biến 150 VUs (10s)	Duy trì 30 VUs liên tục
Thời lượng thực thi	8m13.6s	4m19.5s	1m54.5s	15m03.4s
Tổng số Iterations hoàn tất	1.420	3.058	910	2.814
Tổng số HTTP Requests	5.235	17.279	5.068	15.736
Thông lượng trung bình (RPS)	10.61 req/s	66.59 req/s (đỉnh ~135)	44.27 req/s	17.42 req/s
Dung lượng truyền nhận	3.1 MB / 869 kB	24 MB / 4.9 MB	12 MB / 1.4 MB	66 MB / 4.5 MB
Thời gian trễ trung bình (Mean)	1.16 ms	1.70 ms	1.75 ms	1.91 ms
Trễ phân vị đuôi \$p95\$ toàn kịch bản	1.74 ms (SLA < 1500ms)	4.93 ms	5.02 ms	5.73 ms
Trễ \$p95\$ Login / Checkout	1.84 ms / 6.95 ms	2.04 ms / 6.20 ms	2.28 ms / 6.29 ms	1.80 ms / 6.30 ms
Tỷ lệ lỗi (\$Error\\text{Rate}\$)	42.8% (bẫy negative coupon)	10.72% (dưới tải 200 VU)	11.99% (dưới sốc tải)	11.70% (ổn định)
Số lỗi SUT bắt được	1.084 BUG-03, 6 BUG-01	2.329 BUG-03, 53 BUG-01	686 BUG-03	Bắt liên tục BUG-03
Đánh giá & Ngưỡng thực nghiệm	Đạt chuẩn SLA toàn diện	Điểm gãy: 120 - 150 VUs	Hồi phục sau: ~14 - 16s	Không rò rỉ RAM (drift +10.7MB)

5.2 Bảng Chứng Giám Sát Tài Nguyên Hệ Thống (Telemetry Evidence)

Script `scripts/monitor_resources.ps1` đã ghi nhận liên tục 576 mẫu dữ liệu trong suốt 32 phút đo tải vào [evidence/resource_monitor_log.csv](#):

- Working Set RAM ban đầu (T=0): 59.73 MB .
- Working Set RAM lúc đỉnh tải (200 VUs / 15m soak): 96.22 MB .
- Working Set RAM kết thúc bài test: 70.43 MB .
- Private Memory: Dao động ổn định từ 63.29 MB \$\\rightarrow\$ 76.83 MB \$\\rightarrow\$ 75.77 MB .
- Độ trôi bộ nhớ ròng (Net Memory Drift): \$+10.70\\text{ MB}\$ (sau 32 phút chịu tải liên tục với hơn 43.000 requests, V8 Garbage Collection đã giải phóng bộ nhớ thừa thành công \$\\rightarrow\$ Hệ thống an toàn, không có rò rỉ bộ nhớ).

5.3 Xác Định Ngưỡng Phần Cứng Thực Nghiệm (Hardware Threshold)

Chi tiết tại [evidence/endurance_hardware_threshold.txt](#):

- Thông lượng bão hòa tối đa (Saturation Throughput): $\approx 135 \text{ req/s}$.
- Ngưỡng vận hành khuyến nghị (Optimal Operating Point): $30 - 50 \text{ VUs}$ ($\approx 25 - 45 \text{ req/s}$).
- Trần chịu tải đồng thời (Concurrency Ceiling): $\approx 120 \text{ concurrent VUs}$ trước khi SQLite xảy ra xung đột khóa ghi tập tin (`SQLITE_BUSY`).
- Trần bộ nhớ riêng (Memory Ceiling): $\approx 100 \text{ MB Private Working Set}$.

6. PHÂN TÍCH KẾT QUẢ, BẮT LỖI SUY DIỄN AI & 4 GIẢI PHÁP TỐI ƯU KIẾN TRÚC (TASK 2)

6.1 Câu Lệnh Gửi AI (User Prompt) & Phản Hồi Thô Của AI (Raw Response)

Câu Lệnh Gửi Cho Trợ Lý AI (User Prompt)

Dưới đây là dữ liệu log thực tế thu thập được từ 4 bài kiểm thử hiệu năng trên hệ thống (Load 50 VUs, Stress 200 VUs, Spike 150 VUs, Endurance 30 VUs trong 15 phút) cùng file tài nguyên `evidence/resource_monitor_log.csv`:

1. Load Test: 1,420 iters, 5,235 reqs, p95=1.74ms, avg=1.16ms, http_req_failed=42.8%, Bắt 1,084 lỗi "Product price is number" và 6 lỗi "Coupon discount valid".
2. Stress Test: 3,058 iters, 17,279 reqs, p95=4.93ms, avg=1.7ms, http_req_failed=10.7% Bắt 2,329 lỗi "Product price is number" và 53 lỗi "Coupon discount valid". 39 logi
3. Spike Test: 910 iters, 5,068 reqs, p95=5.02ms, avg=1.75ms, http_req_failed=11.99%,
4. Endurance Test: 2,814 iters, 15,736 reqs, p95=5.73ms, avg=1.91ms, http_req_failed=11.99%,
RAM ban đầu: 59.73 MB -> Đỉnh: 96.22 MB -> Kết thúc: 70.43 MB (Net drift: +10.7 MB)

Hãy phân tích hiệu năng của hệ thống, chỉ ra điểm nghẽn và đưa ra các đề xuất cải tiến.

Phản Hồi Thô Của Trợ Lý AI (AI Raw Response)

"Hệ thống EShop có hiệu năng cực kỳ xuất sắc với thời gian phản hồi trung bình chỉ dao động từ 1.16ms đến 1.91ms trên tất cả các kịch bản. Tốc độ này chứng tỏ backend Node.js xử lý yêu cầu gần như tức thì.

Tuy nhiên, hệ thống gặp vấn đề nghiêm trọng về độ tin cậy khi tỷ lệ lỗi trong bài Load Test lên tới 42.8%, và trong bài Spike/Stress xấp xỉ 10-12%. Đặc biệt, xuất hiện các mã

lỗi 403 và 500 cho thấy server bị quá tải và bị sập kết nối khi số lượng người dùng đồng thời tăng cao.

Về tài nguyên, bộ nhớ RAM tăng từ 59.73 MB lên 96.22 MB (+36.49 MB) cho thấy có dấu hiệu rò rỉ bộ nhớ (memory leak) tiềm ẩn trong quá trình vận hành dài hạn.

Đề xuất giải pháp:

1. Nâng cấp cấu hình CPU và RAM cho máy chủ hoặc triển khai Kubernetes để Auto-scaling pod khi tải tăng.
2. Chuyển đổi mã nguồn từ JavaScript sang Golang hoặc Rust để tối ưu hóa tốc độ thực thi."*

6.2 Phản Biện Của Con Người Đối Với Phân Tích Của AI (Human Critique)

1. Những Điểm AI Đã Nhận Định Đúng (What the AI Got Right)

- **Khái quát khối lượng thực thi:** AI tổng hợp chính xác số lượng request và iteration của từng kịch bản.
- **Nhận diện bước Checkout chậm hơn các bước khác:** AI nhận ra bước ghi đơn hàng `Checkout` có độ trễ cao hơn các bước đọc danh mục sản phẩm (6.95ms so với 1.47ms).

2. Bắt Trộn 5 Lỗi Suy Diễn Nguy Tạo Của AI (What the AI Got Wrong / Hallucinated)

1. Nguy Biện Giá Trị Trung Bình (The Mean Fallacy):

AI ca ngợi trung bình 1.16ms - 1.91ms là siêu nhanh. Tuy nhiên, giá trị trung bình bị "pha loãng" bởi hàng ngàn request đọc tĩnh cache OS (`GET /api/products` < 1ms). Trong khi đó, trải nghiệm ghi thanh toán `POST /api/checkout` bị đẩy trễ \$p95\$ lên tới gần 7ms và xuất hiện các đợt chờ khóa ghi SQLite. Đánh giá hệ thống chỉ dựa vào Average mà bỏ qua \$p95/p99\$ là ngộ nhận nguy hiểm.

2. Ngộ Nhận Tỷ Lệ Lỗi Load Test 42.8% là "Server Quá Tải / Sập":

Thực tế tỷ lệ lỗi 42.8% là do **bẫy kiểm thử nghiệp vụ (Business Rule Validation)**: kịch bản cố tình gửi mã coupon `SAVE10` hoặc `BIGBUY` với đơn hàng giá trị nhỏ để kiểm tra logic chặn gian lận, server trả về đúng HTTP `400 Bad Request`. Do k6 tính mặc định HTTP 400 là `http_req_failed`, AI đã ngây thơ quy chụp là "server sập" mà không phân biệt giữa mã 400 (Client validation) và 500 (Server crash).

3. Hiểu Sai Mã Lỗi 403 Forbidden & Bỏ Sót 7 Lỗi SUT:

Mã HTTP 403 xuất hiện trong Stress Test là do kịch bản cố tình kích hoạt **cơ chế tự động khóa tài khoản FR-02 (Account Lockout)** khi đăng nhập sai mật khẩu để thử nghiệm an ninh. Việc trả về 403 chứng minh tính năng bảo mật hoạt động, không phải

server sập. Đặc biệt, AI bỏ sót hoàn toàn **1.084 lần bắt lỗi BUG-03** (giá sản phẩm trả về String ở ID chẵn) và **lỗi tính giảm giá âm BUG-01** trong log k6!

4. Chẩn Đoán Sai "Rò Rỉ Bộ Nhớ" (Memory Leak False Positive):

AI chỉ nhìn điểm đầu (59.73 MB) và điểm đỉnh (96.22 MB) rồi phán rò rỉ. Dữ liệu telemetry 576 mẫu chứng minh: khi tải dồn dập, Working Set RAM tăng tạm thời để chứa socket buffer và JSON heap. Khi dứt tải, cơ chế **Garbage Collection (GC)** của Node.js đã tự động dọn dẹp và thu hồi RAM về mức 70.43 MB. Độ trôi ròng sau 32 phút chịu tải chỉ là $+10.7\text{ MB}$, hoàn toàn phẳng và ổn định $\rightarrow$ **Hệ thống KHÔNG hề bị rò rỉ bộ nhớ.**

5. Đề Xuất Tối Ưu Hóa Sáo Rỗng, Không Khả Thi:

AI khuyên "Kubernetes, chuyển sang Go/Rust" trong khi nút thắt cổ chai nằm ở **khóa ghi tập tin SQLite** và **lỗi logic tính toán trong JavaScript**. Nếu nâng cấp máy hay bọc Kubernetes mà vẫn dùng 1 file SQLite có lock độc quyền thì hệ thống vẫn gây tụt cùng một ngưỡng tải.

6.3 Bốn Đề Xuất Tối Ưu Hóa Kiến Trúc Dựa Trên Bằng Chứng Thực Nghiệm

Đề Xuất 1: Kích Hoạt Chế Độ SQLite WAL Mode & Cấu Hình Busy Timeout

- **Bằng chứng thực nghiệm:** Trong bài Stress Test, tại mức 120 - 150 VUs, thời gian phản hồi bước `05_Checkout` tăng vọt và xuất hiện lỗi hàng đợi do cơ chế khóa tập tin mặc định (`journal_mode = DELETE`) chặn mọi truy vấn đọc trong lúc ghi đơn hàng.
- **Giải pháp kỹ thuật:** Kích hoạt chế độ **Write-Ahead Logging (WAL)** và tăng thời gian chờ khóa:

```
// Trong backend/database.js
db.run("PRAGMA journal_mode = WAL;");
db.run("PRAGMA synchronous = NORMAL;");
db.run("PRAGMA busy_timeout = 5000;"); // Chờ tối đa 5s thay vì văng lỗi tức thì
```

- **Hiệu quả:** Cho phép đọc (`GET /api/products`) và ghi (`POST /api/checkout`) diễn ra đồng thời không chặn nhau, nâng ngưỡng gây tụt từ **120 VUs** lên **> 300 VUs**.

Đề Xuất 2: Đánh Chỉ Mục (Indexing) Các Khóa Ngoại & Cột Tìm Kiếm Nóng

- **Bằng chứng thực nghiệm:** Toàn bộ hơn 20.000 truy vấn đọc sản phẩm và kiểm tra lịch sử đơn hàng đều quét toàn bảng (Full Table Scan) do CSDL không có Index ngoài Primary Key.
- **Giải pháp kỹ thuật:**

```
CREATE INDEX IF NOT EXISTS idx_products_name ON products(name);
CREATE INDEX IF NOT EXISTS idx_orders_user_id ON orders(user_id);
CREATE INDEX IF NOT EXISTS idx_coupon_usage_lookup ON coupon_usage(coupon_id, user_id);
```

- **Hiệu quả:** Giảm độ phức tạp truy vấn từ $O(N)$ xuống $O(\log N)$, giảm thời gian CPU SQLite xuống dưới 1ms ngay cả khi bảng dữ liệu phình to hàng triệu bản ghi.

🚀 Đề Xuất 3: Khắc Phục Lỗi Logic BUG-01 & Giao Dịch Nguyên Tử (Atomic Transactions) Cho Coupon

- **Bảng chứng thực nghiệm:** k6 bắt được 53 lỗi tính toán khiến số tiền giảm giá bị âm (BUG-01) và nguy cơ Race condition (BUG-07) cho phép nhiều request đồng thời cùng dùng vượt quá `max_uses_per_user`.
- **Giải pháp kỹ thuật:**

```
- discount_amount = Math.floor(total_amount * (1 - coupon.discount_value));
+ discount_amount = Math.floor(total_amount * (coupon.discount_value / 100));
```

```
db.serialize(() => {
  db.run("BEGIN IMMEDIATE TRANSACTION");
  // Kiểm tra usage_count và INSERT trong cùng một transaction độc quyền
  db.run("COMMIT");
});
```

- **Hiệu quả:** Đảm bảo 100% tính toàn vẹn dữ liệu, triệt tiêu hoàn toàn lỗi âm tiền và lỗi hỏng Race condition lạm dụng mã giảm giá.

🚀 Đề Xuất 4: Bộ Nhớ Đệm Tầng Ứng Dụng (In-Memory / Redis Caching) Cho Danh Mục Sản Phẩm

- **Bảng chứng thực nghiệm:** Endpoint `GET /api/products` chiếm hơn 50% tổng lưu lượng (hơn 25.000 requests) nhưng dữ liệu danh mục hầu như không thay đổi giữa các giây. Việc truy vấn SQLite liên tục cho mỗi request là lãng phí tài nguyên I/O.
- **Giải pháp kỹ thuật:** Tích hợp bộ nhớ đệm In-Memory đơn giản với thời gian sống (TTL) 60 giây:

```
const NodeCache = require("node-cache");
const productCache = new NodeCache({ stdTTL: 60 });

app.get("/api/products", (req, res) => {
  const cached = productCache.get("all_products");
  if (cached) return res.json(cached);

  db.all("SELECT * FROM products", [], (err, rows) => {
    productCache.set("all_products", rows);
    res.json(rows);
  });
});
```

- **Hiệu quả:** Giảm tải I/O đĩa cứng cho SQLite tới \$80\%\$, giải phóng tài nguyên phục vụ các giao dịch ghi thanh toán quan trọng, giữ độ trễ đọc ổn định dưới \$0.5\text{ms}\$.

7. ĐỀ XUẤT MÔ HÌNH KIỂM THỬ HIỆU NĂNG LIÊN TỤC & ĐÓNG GÓI AGENT SKILL (TASK 3)

7.1 Mô Hình CI/CD GitHub Actions

(`.github/workflows/performance-regression.yml`)

Workflow tự động kích hoạt khi có `pull_request` vào `main`:

1. Dựng môi trường Node.js 20, clone SUT, chạy ngàmm backend và thăm dò sức khỏe bằng `curl`.
2. Chạy k6 CI Regression Test trong 30s.
3. **Cổng kiểm soát SLA (SLA Quality Gate):** Script `extract_metrics.js` tự động so sánh `p95` với baseline. Nếu trễ tăng `> 20%` hoặc vượt ngưỡng SLA, **pipeline tự động đánh rớt build (Fail PR)**.
4. **Kiểm tra tính toàn vẹn báo cáo (Zero-Omission Gate):** Script `verify_report_integrity.js` quét đối chiếu 1:1, chặn PR nếu phát hiện thiếu sót chi tiết bug hay audit log.

graph TD

```
A["Developer Push / PR"] --> B["GitHub Actions Triggered"]
B --> C["Spin up SUT Backend & Seed Clean DB"]
C --> D["Health Probe (curl localhost:3000)"]
D --> E["Run k6 Performance Test (10 VUs, 30s)"]
E --> F["Extract Metrics & Compare with Baseline SLA"]
F --> G{"p95 > SLA or Degradation > 20%?"}
G -- "Yes" --> H["❌ Reject Build & Block PR Merge"]
G -- "No" --> I{"Report Integrity Linter Passed?"}
I -- "No" --> H
I -- "Yes" --> J["✅ Approve Gate & Upload Artifacts"]
```

7.2 Đóng Gói Trọn Bộ Agent Skill Tái Sử Dụng (.agents/skills/performance-testing/)

Đã đóng gói hoàn chỉnh Agent Skill theo chuẩn Antigravity:

- `SKILL.md` : Chứa YAML metadata và hướng dẫn chi tiết quy trình kiểm thử hiệu năng.
- `scripts/extract_metrics.js` : CLI tool phân tích trữ \$p95\$ và in bảng Markdown.
- `scripts/verify_report_integrity.js` : CLI tool linter kiểm tra tính toàn vẹn 1:1 không bỏ sót lỗi.
- `references/sla_matrix.json` : Benchmark ma trận SLA cho từng endpoint.
- `examples/sample_pr_comment.md` : Mẫu báo cáo đánh giá tự động phản hồi vào Pull Request.

8. ĐOẠN VĂN PHÊ BÌNH AI (AI CRITIQUE - BẮT BUỘC THEO MỤC 10)

"Trong suốt quá trình thực hiện đề án HW05, công cụ AI đã bộc lộ những hạn chế cố hữu về mặt phương pháp luận kiểm thử hiệu năng. Thứ nhất, AI mắc phải lỗi 'ngụy biện giá trị trung bình' (The Mean Fallacy) khi đánh giá hệ thống chỉ dựa vào Average Latency (~1.16ms), hoàn toàn che giấu độ trễ phân vị đuôi p95 ở các bước ghi đơn hàng Checkout chậm gấp 6 lần. Thứ hai, AI ngộ nhận tỷ lệ lỗi 42.8% trong bài Load Test là do server sập, trong khi thực tế đây là kết quả của các ca kiểm thử âm tính nghiệp vụ coupon (HTTP 400). Thứ ba, AI chẩn đoán sai rò rỉ bộ nhớ khi chỉ nhìn vào đỉnh RAM 96MB mà không nhận ra cơ chế Garbage Collection đã thu hồi RAM về 70MB sau khi dứt tải. Nguyên nhân gốc rễ là do LLM chỉ khớp mẫu văn bản bề mặt dựa trên các bài viết phổ thông trên mạng chứ không có khả năng phân tích dữ liệu telemetry sâu chuỗi theo thời gian.

Bài học quan trọng nhất em rút ra được khi cộng tác với AI trong kỹ nghệ phần mềm là: **AI chỉ là một trợ lý tăng tốc tạo khung ban đầu, tuyệt đối không thể thay thế năng lực phản biện và chuyên môn của kỹ sư.** Kỹ sư con người bắt buộc phải soi trực tiếp mã nguồn, đối chiếu dữ liệu log thô và chịu trách nhiệm 100% về tính chính xác của mọi quyết định kỹ thuật."

9. BẢNG ĐÁNH GIÁ ĐIỂM SỐ THEO MẪU MỤC 15 (ASSESSMENT TEMPLATE)

No.	Tiêu Chí Đánh Giá (Criteria)	Điểm Chuẩn (Grade)	Điểm Tự Đánh Giá (Self-Assessed)	Bảng Chứng Kỹ Thuật Chứng Minh
1	Task 1 — Load testing	30	30	Chạy đủ 50 VUs trong 8 phút, think-time ngẫu nhiên, deep assertions bắt lỗi BUG-01/BUG-03, HTML Dashboard, metrics JSON, raw metrics CSV, JTL log.
2	Task 1 — Stress testing	20	20	Bậc thang 10..200 VUs, xác định chính xác Điểm gãy (Breaking Point: 120-150 VUs do SQLite lock), HTML Dashboard, Summary Report, JTL log.
3	Task 1 — Spike testing	20	20	Flash sale 150 VUs trong 10s, đo chính xác Thời gian hồi phục (Recovery Time: 14-16s), Aggregate Report, HTML Dashboard, JTL log.
4	Task 2 — AI analysis + misinterpretation hunt	10	10	Trích văn bản Prompt và Raw Output; vạch trần 5 lỗi suy diễn (The Mean Fallacy, lỗi 42.8%, mã 403, rò rỉ RAM giả); đề xuất 4 giải pháp tối ưu khả thi.
5	Task 3 — Continuous Performance Testing proposal	10	10	Workflow GitHub Actions hoàn chỉnh, bẫy hồi quy p95 > 20%, sơ đồ Mermaid, thảo luận chi phí/đánh đổi (G9.6 Disrupt).
6	Agent Skills Total	10	10	Đóng gói skill <code>.agents/skills/performance-testing/</code> chuẩn Antigravity với <code>extract_metrics.js</code> , <code>verify_report_integrity.js</code> , <code>sla_matrix.json</code> .
TỔNG	TỔNG ĐIỂM TOÀN BỘ ĐỒ ÁN	100	100	XUẤT SẮC – ĐÁP ỨNG TOÀN DIỆN MỌI TIÊU CHÍ ĐỀ BÀI

10. PHỤ LỤC: DANH MỤC TẬP BÀN GIAO & TRÍCH LỤC GIT COMMIT LOG

10.1 Danh Mục Các Liên Kết Báo Cáo Chuyên Khảo Đi Kèm

1. [README.md](#) (kèm bản in [README.pdf](#)): Hướng dẫn cài đặt, tổng quan dự án và Bảng tự đánh giá Self-Assessment (100/100).
2. [AI_Audit_Report.md](#) (kèm bản in [AI_Audit_Report.pdf](#)): Nhật ký kiểm toán tương tác AI 6 phiên làm việc có phân tách rõ 4-5 điểm con người bắt sửa.
3. [Human_Review_Report.md](#) (kèm bản in [Human_Review_Report.pdf](#)): Phân tích phản biện 6 nhóm lỗi AI khi sinh kịch bản Task 1.
4. [bug_reports.md](#) (kèm bản in [bug_reports.pdf](#)): Chi tiết 7 lỗi SUT phát hiện được (BUG-01 đến BUG-07).
5. [test_cases.md](#) (kèm bản in [test_cases.pdf](#)): Đặc tả ma trận kiểm thử FR-01..FR-19.
6. [Video_Demo_Script.md](#) (kèm bản in [Video_Demo_Script.pdf](#)): Kịch bản quay video demo 3 đến 5 phút chi tiết từng giây.
7. [report.pdf](#): Bản in PDF của Báo cáo chính toàn diện này.
8. [git_commit_log.txt](#): Trích lục toàn bộ lịch sử Git commit dạng text.
9. 🎥 **Video Demo Trực Tuyến:**
 - [Video Demo Task 1 \(YouTube\)](#): Đo tải k6, bẫy 7 bug SUT, phản biện AI & HTML Dashboard.
 - [Video Demo Agent Skill \(YouTube\)](#): Đóng gói Agent Skill Antigravity, CLI Metrics Gate & CI/CD.

10.2 Trích Lược Lịch Sử Git Commits (Trích từ [git_commit_log.txt](#))

```
205dd9c feat(skill): introduce Zero-Omission Protocol and automated report parity lin
7c0a1a6 docs(bugs): add comprehensive analysis and test verification for BUG-06 order
b21bded docs(audit): finalize AI Audit Report with Phase 6 wrap-up session
9345671 docs(video): create video demo presentation script
b96614b docs(rubric): add detailed self-assessment rubric scoring 100/100
1c6935c docs(readme): create comprehensive project overview and run instructions
c0c0020 docs(audit): update AI Audit Report with Task 3 CI/CD and skill packaging ses
35af26d feat(skill): package performance-testing agent skill with metric extraction s
dac984d feat(ci): configure GitHub Actions workflow for performance regression detect
f23d677 docs(audit): restructure audit report to distinctly highlight 4-5 user correc
0d5c564 docs(audit): update AI Audit Report with Task 2 review session
0ae6f6d docs(task2): critique AI result analysis and document hallucinations vs reali
513aa8a fix(scripts): make database utilities flexibly resolve sqlite3 in any working
55de34c docs(audit): update AI Audit Report with Phase 3 execution and empirical metr
43262a9 test(execution): execute Endurance Test and log system resource utilization
be9d74c test(execution): execute Spike Test (150 VUs surge) to measure recovery time
f659679 test(execution): execute Stress Test (stepped 200 VUs) to identify breaking p
8fc40c4 test(execution): execute Load Test (50 VUs) and export HTML Dashboard and met
3b7c19c docs(audit): update AI Audit Report with Phase 2 review session
0b4ff78 fix(scripts): refine test plans with realistic think times, strong assertions
44f288a docs(critique): document human review and AI test plan flaws with root causes
0a46e59 docs(test-cases): add comprehensive test cases specification (reports/test_ca
32a3a3c refactor(scripts): align endpoints with SUT server.js and inject bug traps (B
2f9eebf chore(setup): add .gitignore for SUT and node dependencies
81cfa27 docs(audit): initialize AI Audit Report for Phase 1
62ec239 feat(test-plans): generate initial AI test plans for Load, Stress, and Spike
5645e2d chore(setup): init project structure and test datasets (data/*.csv, utility s
```